

Gradient Descent for Logistic Regression

A Matrix-Form Derivation & Lecture Note

Prepared for the Machine Learning Course (CSE 403)

Md. Shoab Alam • Department of CSE

Contents

1	Setup: Notation and the Data Matrix	2
2	The Prediction for a Single Example	2
3	Stacking All Predictions: The Vector $\hat{y}$	3
4	The Bias Trick and the Compact Matrix Form	3
5	The Loss Function (Binary Cross-Entropy)	3
5.1	Converting the First Sum to a Dot Product	4
5.2	Loss in Compact Matrix Form	4
6	Gradient Descent	4
6.1	The Update Rule	4
6.2	The Gradient in Matrix Form	5
6.3	Final Weight-Update Equation	5
7	Putting It All Together: Dimensions & Algorithm	5
7.1	Shapes of Every Object	5
7.2	The Training Loop	6
8	A Tiny Numerical Walk-through	6
9	Full Worked Solution: Gradient Descent on a Real Dataset	6
9.1	The Dataset and Initial Settings	7
9.2	Iteration 1 (Epoch 1)	7
9.3	Iteration 2 (Epoch 2)	8
9.4	Iteration 3 (Epoch 3)	9
9.5	Progress Across the Three Iterations	11
10	Summary of Key Formulas	11

1. Setup: Notation and the Data Matrix

Before we can talk about gradient descent, we must fix our notation. In a supervised learning problem we are given a dataset of m training examples, each described by n features.

- **Rows** = m (number of training examples / data points).
- **Columns** = n (number of input features).
- x_{ij} is the value of the j -th feature of the i -th example.
- y_i is the true label of the i -th example, and $\hat{y}_i$ is our prediction.

The full feature matrix looks like this:

$$\underbrace{\begin{bmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1n} \\ x_{21} & x_{22} & x_{23} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & x_{m3} & \cdots & x_{mn} \end{bmatrix}}_{n \text{ features (columns)}} \longleftrightarrow \hat{\mathbf{y}} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

We also have one weight per feature, plus a bias term w_0 :

$$\{w_1, w_2, w_3, \dots, w_n\} \text{ and the bias } w_0.$$

Counting the bias, there are $(n + 1)$ parameters in total.

Intuition

Think of each **row** as one student, and each **column** as one piece of information about that student (marks, attendance, etc.). The label y_i is what we want to predict (e.g. pass = 1, fail = 0).

2. The Prediction for a Single Example

Logistic regression first computes a weighted sum of the inputs (a linear combination), then squashes it through the **sigmoid** function $\sigma(z) = \frac{1}{1 + e^{-z}}$ to obtain a probability between 0 and 1.

For example, for the first and second training rows:

$$\sigma(w_1x_{11} + w_2x_{12} + w_3x_{13} + \cdots + w_nx_{1n} + w_0) = \hat{y}_1,$$

$$\sigma(w_1x_{21} + w_2x_{22} + w_3x_{23} + \cdots + w_nx_{2n} + w_0) = \hat{y}_2.$$

In general, for row i :

$$\hat{y}_i = \sigma\left(w_0 + \sum_{j=1}^n w_j x_{ij}\right)$$

3. Stacking All Predictions: The Vector $\hat{y}$

If we write the prediction for every row and stack them into a column vector, we obtain:

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \sigma(w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_nx_{1n}) \\ \sigma(w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_nx_{2n}) \\ \vdots \\ \sigma(w_0 + w_1x_{m1} + w_2x_{m2} + \cdots + w_nx_{mn}) \end{bmatrix}$$

Because the sigmoid is applied element-wise, we can pull it outside the vector:

$$\hat{y} = \sigma \left(\begin{bmatrix} w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_nx_{1n} \\ w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_nx_{2n} \\ \vdots \\ w_0 + w_1x_{m1} + w_2x_{m2} + \cdots + w_nx_{mn} \end{bmatrix} \right)$$

4. The Bias Trick and the Compact Matrix Form

The expression inside the sigmoid is exactly a matrix–vector product. To absorb the bias w_0 cleanly, we prepend a **column of ones** to the feature matrix. This is called the *bias trick*.

$$\hat{y} = \sigma \left(\underbrace{\begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}}_{X \ (m \times (n+1))} \underbrace{\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}}_{W \ ((n+1) \times 1)} \right)$$

Here the leading column of 1's multiplies w_0 , so every row automatically includes the bias term. Naming the augmented feature matrix X and the weight vector W , the whole model collapses into one elegant line:

Key Result — Model in Matrix Form

$$\hat{y} = \sigma(XW)$$

Intuition

Dimension check. X is $m \times (n+1)$ and W is $(n+1) \times 1$, so XW is $m \times 1$ — exactly one score per training example, which is what we want.

5. The Loss Function (Binary Cross-Entropy)

To measure how wrong our predictions are, logistic regression uses the **binary cross-entropy** (log loss). Summed over all m examples:

$$L = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

We can split the single sum into two sums:

$$L = -\frac{1}{m} \left[\sum_{i=1}^m y_i \log(\hat{y}_i) + \sum_{i=1}^m (1 - y_i) \log(1 - \hat{y}_i) \right]$$

5.1 Converting the First Sum to a Dot Product

Expand the first summation term by term:

$$\sum_{i=1}^m y_i \log(\hat{y}_i) = y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2 + y_3 \log \hat{y}_3 + \cdots + y_m \log \hat{y}_m.$$

This is precisely the dot product of the label (row) vector with the $\log(\hat{y})$ (column) vector:

$$\sum_{i=1}^m y_i \log(\hat{y}_i) = \begin{bmatrix} y_1 & y_2 & y_3 & \cdots & y_m \end{bmatrix} \begin{bmatrix} \log \hat{y}_1 \\ \log \hat{y}_2 \\ \vdots \\ \log \hat{y}_m \end{bmatrix}$$

Pulling the log outside the vector and writing $\hat{y} = \sigma(XW)$:

$$\begin{bmatrix} y_1 & y_2 & y_3 & \cdots & y_m \end{bmatrix} \log \left(\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} \right) \implies \boxed{y \log \hat{y} = y \log(\sigma(XW))}$$

5.2 Loss in Compact Matrix Form

Applying the same idea to the second sum, the entire loss becomes:

Key Result — Loss in Matrix Form

$$L = -\frac{1}{m} \left[y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \right], \quad \text{where } \hat{y} = \sigma(XW).$$

Intuition

Every symbol here ($y, \hat{y}$) is now a *vector*, and the products are *vector/matrix* operations. This is why the matrix form is so powerful: the same compact formula handles all m examples at once, with no explicit loops.

6. Gradient Descent

Our goal is to find the weights W that make the loss L as small as possible (the **minimum**). Gradient descent does this by repeatedly stepping in the direction that decreases L the fastest — the negative gradient.

6.1 The Update Rule

For each pass over the data (an **epoch**) we update every weight:

$$\boxed{W = W - \eta \frac{\partial L}{\partial W}}$$

where η (eta) is the **learning rate** — a small positive number controlling the step size.

The gradient itself is a vector containing one partial derivative per weight, from w_0 up to w_n (that is $(n + 1)$ entries):

$$\frac{\partial L}{\partial W} = \begin{bmatrix} \frac{\partial L}{\partial w_0} & \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_2} & \cdots & \frac{\partial L}{\partial w_n} \end{bmatrix}$$

6.2 The Gradient in Matrix Form

Working through the calculus of the cross-entropy loss (the sigmoid derivative makes the algebra cancel beautifully) gives a remarkably clean result:

Key Result — Gradient

$$\frac{\partial L}{\partial W} = -\frac{1}{m} (y - \hat{y}) X$$

6.3 Final Weight-Update Equation

Substituting the gradient into the update rule and simplifying the two negative signs ($-\eta \times -\frac{1}{m} = +\frac{\eta}{m}$):

Key Result — Gradient Descent Update

$$W = W + \eta \frac{1}{m} (y - \hat{y}) X$$

Intuition

Why the sign flips. The gradient carries a minus sign, and the update rule subtracts the gradient. Two minuses make a plus, so the final update *adds* the term. The factor $(y - \hat{y})$ is simply the **error**: how far our prediction is from the truth.

7. Putting It All Together: Dimensions & Algorithm

7.1 Shapes of Every Object

Object	Symbol	Shape
Weight vector	W	$(n + 1) \times 1$
Augmented features	X	$m \times (n + 1)$
True labels	y	$m \times 1$
Predictions	$\hat{y}$	$m \times 1$

For completeness, the three core vectors/matrices are:

$$W = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & & & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}.$$

7.2 The Training Loop

1. **Initialize** the weight vector W (e.g. zeros or small random values).
2. **For each epoch** (repeat until convergence):
 - (a) Compute predictions: $\hat{y} = \sigma(XW)$.
 - (b) Compute the gradient: $\frac{\partial L}{\partial W} = -\frac{1}{m}(y - \hat{y})X$.
 - (c) Update the weights: $W = W + \eta \frac{1}{m}(y - \hat{y})X$.
3. **Stop** when the loss L stops decreasing meaningfully.

8. A Tiny Numerical Walk-through

Worked Example

Suppose we have $m = 2$ examples and $n = 1$ feature. After the bias trick:

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad W = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \eta = 0.1.$$

Step 1 — Linear scores.

$$XW = \begin{bmatrix} 1 \cdot 0 + 2 \cdot 0 \\ 1 \cdot 0 + 3 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Step 2 — Predictions through sigmoid. Since $\sigma(0) = 0.5$,

$$\hat{y} = \sigma(XW) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}.$$

Step 3 — Error.

$$y - \hat{y} = \begin{bmatrix} 0 - 0.5 \\ 1 - 0.5 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix}.$$

Step 4 — Gradient ($\frac{\partial L}{\partial W} = -\frac{1}{m}(y - \hat{y})^\top X$):

$$-\frac{1}{2} \begin{bmatrix} -0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} = -\frac{1}{2} \begin{bmatrix} 0 & 0.5 \end{bmatrix} = \begin{bmatrix} 0 & -0.25 \end{bmatrix}.$$

Step 5 — Update ($W = W - \eta \frac{\partial L}{\partial W}$):

$$W = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 0 \\ -0.25 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.025 \end{bmatrix}.$$

The weight w_1 moved in the positive direction — exactly what we expect, since the example with the larger feature value had label 1. Repeating this loop many times drives the loss down.

9. Full Worked Solution: Gradient Descent on a Real Dataset

We now apply gradient descent for a few iterations on a concrete dataset, so every formula above becomes a number you can check by hand.

9.1 The Dataset and Initial Settings

Student ID	x_0 (bias)	x_1	y
1	1	0.5	0
2	1	1.5	0
3	1	2.5	1
4	1	3.0	1
5	1	1.0	0

- Number of training examples: $m = 5$.
- Bias term: $x_0 = 1$ for every student (the bias trick column).
- Initial parameters: $w_0 = 0.0$ and $w_1 = 0.0$.
- Learning rate: $\eta = 0.1$.

In matrix form the data is:

$$X = \begin{bmatrix} 1 & 0.5 \\ 1 & 1.5 \\ 1 & 2.5 \\ 1 & 3.0 \\ 1 & 1.0 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \quad W = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Recall the three formulas we will use each iteration:

$$\hat{y} = \sigma(XW), \quad \frac{\partial L}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i) x_{ij}, \quad w_j = w_j - \eta \frac{\partial L}{\partial w_j}.$$

Intuition

How to read the sigmoid step. To compute $\sigma(z) = \frac{1}{1 + e^{-z}}$ by hand, first find e^{-z} , then add 1, then take the reciprocal. For example with $z = 0.0275$: $e^{-0.0275} = 0.972875$, so $\sigma(0.0275) = \frac{1}{1.972875} = 0.506875$. Every prediction below is obtained exactly this way.

9.2 Iteration 1 (Epoch 1)

Step 1 — Linear score $z_i = w_0 + w_1 x_{i1}$.

Since $w_0 = w_1 = 0$, every score is 0:

$$z = [0 \quad 0 \quad 0 \quad 0 \quad 0]^\top.$$

Step 2 — Predictions $\hat{y}_i = \sigma(z_i)$.

Because $\sigma(0) = \frac{1}{1 + e^0} = 0.5$:

$$\hat{y} = [0.5 \quad 0.5 \quad 0.5 \quad 0.5 \quad 0.5]^\top.$$

Step 3 — Errors $(y_i - \hat{y}_i)$.

$$y - \hat{y} = \begin{bmatrix} 0 - 0.5 \\ 0 - 0.5 \\ 1 - 0.5 \\ 1 - 0.5 \\ 0 - 0.5 \end{bmatrix} = \begin{bmatrix} -0.5 \\ -0.5 \\ 0.5 \\ 0.5 \\ -0.5 \end{bmatrix}.$$

Step 4 — Gradients. Using $\frac{\partial L}{\partial w_j} = -\frac{1}{m} \sum_i (y_i - \hat{y}_i) x_{ij}$.

For w_0 (here $x_{i0} = 1$):

$$\frac{\partial L}{\partial w_0} = -\frac{1}{5}(-0.5 - 0.5 + 0.5 + 0.5 - 0.5) = -\frac{1}{5}(-0.5) = 0.1.$$

For w_1 (multiply each error by its x_{i1}):

$$\begin{aligned} \sum_i (y_i - \hat{y}_i) x_{i1} &= (-0.5)(0.5) + (-0.5)(1.5) + (0.5)(2.5) + (0.5)(3.0) + (-0.5)(1.0) \\ &= -0.25 - 0.75 + 1.25 + 1.5 - 0.5 = 1.25, \quad \frac{\partial L}{\partial w_1} = -\frac{1}{5}(1.25) = -0.25. \end{aligned}$$

Step 5 — Update $w_j = w_j - \eta \partial L / \partial w_j$.

$$w_0 = 0 - 0.1(0.1) = -0.01, \quad w_1 = 0 - 0.1(-0.25) = 0.025.$$

Step 6 — Loss for this epoch. Using $L = -\frac{1}{m} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$. With every $\hat{y}_i = 0.5$, each example contributes the same amount. For a label $y_i = 0$ the term is $\log(1 - 0.5) = \log(0.5) = -0.693147$; for $y_i = 1$ it is $\log(0.5) = -0.693147$ as well. So all five terms equal -0.693147 :

$$L = -\frac{1}{5}(-0.693147 \times 5) = -\frac{1}{5}(-3.465736) = 0.693147.$$

After Epoch 1

$$w_0 = -0.0100, \quad w_1 = 0.0250, \quad L \approx 0.6931.$$

Intuition

w_1 increased because the students with larger x_1 values (IDs 3 and 4) are the positive examples. Gradient descent pushes the weight to give larger inputs a higher predicted probability.

9.3 Iteration 2 (Epoch 2)

We now use the *updated* weights $w_0 = -0.01$, $w_1 = 0.025$. Nothing is skipped — every score, sigmoid, error and product is shown exactly as in Epoch 1.

Step 1 — Linear scores $z_i = w_0 + w_1 x_{i1}$.

$$\begin{aligned} z_1 &= -0.01 + 0.025(0.5) = 0.002500, \\ z_2 &= -0.01 + 0.025(1.5) = 0.027500, \\ z_3 &= -0.01 + 0.025(2.5) = 0.052500, \\ z_4 &= -0.01 + 0.025(3.0) = 0.065000, \\ z_5 &= -0.01 + 0.025(1.0) = 0.015000. \end{aligned}$$

Step 2 — Predictions $\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$.

$$\begin{aligned} \hat{y}_1 &= \frac{1}{1 + e^{-0.0025}} = \frac{1}{1 + 0.997503} = 0.500625, \\ \hat{y}_2 &= \frac{1}{1 + e^{-0.0275}} = \frac{1}{1 + 0.972875} = 0.506875, \\ \hat{y}_3 &= \frac{1}{1 + e^{-0.0525}} = \frac{1}{1 + 0.948854} = 0.513122, \\ \hat{y}_4 &= \frac{1}{1 + e^{-0.0650}} = \frac{1}{1 + 0.937067} = 0.516244, \\ \hat{y}_5 &= \frac{1}{1 + e^{-0.0150}} = \frac{1}{1 + 0.985112} = 0.503750. \end{aligned}$$

Step 3 — Errors ($y_i - \hat{y}_i$).

$$\begin{aligned} e_1 &= 0 - 0.500625 = -0.500625, & e_2 &= 0 - 0.506875 = -0.506875, \\ e_3 &= 1 - 0.513122 = 0.486878, & e_4 &= 1 - 0.516244 = 0.483756, \\ e_5 &= 0 - 0.503750 = -0.503750. \end{aligned}$$

Step 4a — Gradient w.r.t. w_0 (each $x_{i0} = 1$, so just sum the errors):

$$\sum_i e_i = -0.500625 - 0.506875 + 0.486878 + 0.483756 - 0.503750 = -0.540616,$$

$$\frac{\partial L}{\partial w_0} = -\frac{1}{5}(-0.540616) = 0.108123.$$

Step 4b — Gradient w.r.t. w_1 (multiply each error by its x_{i1}):

$$\begin{aligned} e_1 x_{11} &= -0.500625(0.5) = -0.250312, \\ e_2 x_{21} &= -0.506875(1.5) = -0.760312, \\ e_3 x_{31} &= 0.486878(2.5) = 1.217195, \\ e_4 x_{41} &= 0.483756(3.0) = 1.451267, \\ e_5 x_{51} &= -0.503750(1.0) = -0.503750. \end{aligned}$$

$$\sum_i e_i x_{i1} = 1.154088, \quad \frac{\partial L}{\partial w_1} = -\frac{1}{5}(1.154088) = -0.230818.$$

Step 5 — Update $w_j = w_j - \eta \partial L / \partial w_j$.

$$\begin{aligned} w_0 &= -0.010000 - 0.1(0.108123) = -0.020812, \\ w_1 &= 0.025000 - 0.1(-0.230818) = 0.048082. \end{aligned}$$

Step 6 — Loss. Per-example terms $\ell_i = y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$:

$$\ell_1 = -0.694398, \ell_2 = -0.706992, \ell_3 = -0.667242, \ell_4 = -0.661175, \ell_5 = -0.700675.$$

$$L = -\frac{1}{5}(-3.430482) = 0.686096.$$

After Epoch 2

$$w_0 \approx -0.020812, \quad w_1 \approx 0.048082, \quad L \approx 0.686096.$$

9.4 Iteration 3 (Epoch 3)

Now use $w_0 = -0.020812$, $w_1 = 0.048082$.

Step 1 — Linear scores.

$$\begin{aligned} z_1 &= -0.020812 + 0.048082(0.5) = 0.003229, \\ z_2 &= -0.020812 + 0.048082(1.5) = 0.051310, \\ z_3 &= -0.020812 + 0.048082(2.5) = 0.099392, \\ z_4 &= -0.020812 + 0.048082(3.0) = 0.123433, \\ z_5 &= -0.020812 + 0.048082(1.0) = 0.027269. \end{aligned}$$

Step 2 — Predictions $\hat{y}_i = \sigma(z_i)$.

$$\begin{aligned}\hat{y}_1 &= \frac{1}{1+e^{-0.003229}} = 0.500807, \\ \hat{y}_2 &= \frac{1}{1+e^{-0.051310}} = 0.512825, \\ \hat{y}_3 &= \frac{1}{1+e^{-0.099392}} = 0.524828, \\ \hat{y}_4 &= \frac{1}{1+e^{-0.123433}} = 0.530819, \\ \hat{y}_5 &= \frac{1}{1+e^{-0.027269}} = 0.506817.\end{aligned}$$

Step 3 — Errors $(y_i - \hat{y}_i)$.

$$\begin{aligned}e_1 &= 0 - 0.500807 = -0.500807, & e_2 &= 0 - 0.512825 = -0.512825, \\ e_3 &= 1 - 0.524828 = 0.475172, & e_4 &= 1 - 0.530819 = 0.469181, \\ e_5 &= 0 - 0.506817 = -0.506817.\end{aligned}$$

Step 4a — Gradient w.r.t. w_0 .

$$\sum_i e_i = -0.500807 - 0.512825 + 0.475172 + 0.469181 - 0.506817 = -0.576096,$$

$$\frac{\partial L}{\partial w_0} = -\frac{1}{5}(-0.576096) = 0.115219.$$

Step 4b — Gradient w.r.t. w_1 .

$$\begin{aligned}e_1 x_{11} &= -0.500807(0.5) = -0.250404, \\ e_2 x_{21} &= -0.512825(1.5) = -0.769237, \\ e_3 x_{31} &= 0.475172(2.5) = 1.187931, \\ e_4 x_{41} &= 0.469181(3.0) = 1.407543, \\ e_5 x_{51} &= -0.506817(1.0) = -0.506817.\end{aligned}$$

$$\sum_i e_i x_{i1} = 1.069016, \quad \frac{\partial L}{\partial w_1} = -\frac{1}{5}(1.069016) = -0.213803.$$

Step 5 — Update.

$$\begin{aligned}w_0 &= -0.020812 - 0.1(0.115219) = -0.032334, \\ w_1 &= 0.048082 - 0.1(-0.213803) = 0.069462.\end{aligned}$$

Step 6 — Loss. Per-example terms:

$$\ell_1 = -0.694763, \ell_2 = -0.719131, \ell_3 = -0.644685, \ell_4 = -0.633334, \ell_5 = -0.706875.$$

$$L = -\frac{1}{5}(-3.398788) = 0.679758.$$

After Epoch 3

$$w_0 \approx -0.032334, \quad w_1 \approx 0.069462, \quad L \approx 0.679758.$$

9.5 Progress Across the Three Iterations

Epoch	w_0	w_1	$\partial L / \partial w_0$	$\partial L / \partial w_1$	Loss L
Start	0.000000	0.000000	–	–	–
1	–0.010000	0.025000	0.100000	–0.250000	0.693147
2	–0.020812	0.048082	0.108123	–0.230818	0.686096
3	–0.032334	0.069462	0.115219	–0.213803	0.679758

Intuition

Two things to point out to students: (1) the loss L **decreases** every epoch (0.6931 $\rightarrow$ 0.6861 $\rightarrow$ 0.6798), which confirms the weights are moving in the right direction; and (2) w_1 keeps **growing** while w_0 slowly becomes negative — the model is learning that a higher x_1 means a higher chance of $y = 1$. After many more epochs these values converge to the best-fit logistic boundary.

10. Summary of Key Formulas

Quantity	Formula
Prediction (matrix form)	$\hat{y} = \sigma(XW)$
Loss (matrix form)	$L = -\frac{1}{m} [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
Gradient	$\frac{\partial L}{\partial W} = -\frac{1}{m} (y - \hat{y})X$
Update rule (general)	$W = W - \eta \frac{\partial L}{\partial W}$
Update rule (final)	$W = W + \eta \frac{1}{m} (y - \hat{y})X$

Based on the lecture “Gradient Descent for Logistic Regression.”
Reference video: <https://www.youtube.com/watch?v=ABrrSwMYWSg>